



廣東工業大學

GUANGDONG UNIVERSITY OF TECHNOLOGY

嵌入式系统课程设计个人总结报告

《基于 STM32F103C8T6 的智能手表设计》

姓 名： 郑昭宜

学 号： 3224009583

专 业： 微电子科学与工程

班 级： 微电子 1 班

队 名： 大天才智能手表

队 友： 沈雯洁、何洁瑜

指导老师： 周贤中



集成电路学院

2026 年 7 月 11 日

1 项目简介

1.1 项目背景

本项目旨在设计一款基于 STM32F103 的智能运动健康监测设备。系统需集成传感器数据采集、实时计步、OLED 显示、蓝牙通信及 FreeRTOS 多任务调度等功能，实现对用户运动状态的实时监测与数据同步。

1.2 主要功能概述

作为任务分工里的成员二，我主要负责：

- 基于 **软件 I2C** 协议驱动 MPU6050 六轴传感器，采集加速度与角速度数据。
- 设计并实现 **高精度计步算法**，通过三轴合加速度向量模平方进行动态阈值判断。
- 负责 **FreeRTOS 多任务架构** 中传感器采集与计步处理任务的创建与调度。
- 完成 **MPU6050 与 EEPROM** 的硬件接线及 **焊接加固**，保障系统稳定性。
- 与主控组、蓝牙组联调，实现运动数据的存储、显示与无线传输。

2 项目设计目标

2.1 设计目标

- 实现 MPU6050 传感器的稳定读取，数据刷新率不低于 50Hz。
- 设计抗干扰能力强的计步算法，在常规步行场景下准确率 $> 95\%$ 。
- 构建基于 FreeRTOS 的多任务系统，确保传感器采集、数据处理、显示刷新互不阻塞。
- 完成硬件层面的可靠性加固，消除因震动或接触不良导致的通信故障。

2.2 预期成果

- 一套可运行的软硬件系统，包含完整的传感器驱动与计步功能。
- 课程设计报告文档，涵盖软件 I2C 实现、计步算法原理、FreeRTOS 任务设计。

3 项目设计与实现

3.1 硬件设计

3.1.1 系统架构与模块选型

本模块主要涉及传感与数据存储：

- **MPU6050**：六轴惯性测量单元（加速度计 + 陀螺仪），用于捕捉人体运动姿态。
- **EEPROM**：用于掉电存储历史计步数据（扩展功能）。
- **STM32F103**：主控 MCU，负责数据处理与任务调度。

3.1.2 电路原理图与硬件连接

- MPU6050 通过 **软件 I2C** 接口（GPIO 模拟）与 STM32 连接，SCL 接 PB8，SDA 接 PB9，地址为 0x68。
- EEPROM 通过 I2C 总线复用连接，用于存储累计步数。
- **硬件加固**：针对 MPU6050 模块引脚易松动的问题，我对部分模块与接线的连接点进行了 **手工焊接加固**，显著提升了机械稳定性。

3.2 软件设计

3.2.1 软件架构

系统基于 **FreeRTOS** 构建，采用多任务并发模型：

- **Task_MPU6050**（优先级 3）：负责传感器数据采集与计步算法处理。
- **Task_Display**（优先级 2）：负责 OLED 显示刷新。
- **Task_MenuCtrl**（优先级 1）：负责菜单交互。
- **Task_Bluetooth**（优先级 1）：负责蓝牙数据透传。

任务间通过全局变量 `step_count` 共享计步数据。

3.2.2 关键代码与技术实现细节

1. 软件 I2C 驱动实现 鉴于 STM32F103 硬件 I2C 的“总线挂死”缺陷，我采用 GPIO 模拟 I2C 时序：

- 配置 SCL/SDA 为 **开漏输出 (Open-Drain)**，配合外部 $4.7k\Omega$ 上拉电阻。
- 实现 SDA_Input() / SDA_Output() 动态切换，满足 I2C 双向通信需求。
- 严格遵循时序：起始信号 (START)、停止信号 (STOP)、ACK/NACK 应答机制。
- 提供 Soft_I2C_WriteByte() 与 Soft_I2C_ReadBytes() 接口供上层调用。

2. MPU6050 驱动实现

- 初始化流程**：复位设备 ($0x6B \leftarrow 0x80$) $\rightarrow$ 唤醒 ($0x6B \leftarrow 0x00$) $\rightarrow$ 配置量程 ($\pm 2g$) $\rightarrow$ 关闭 FIFO 与中断。
- 数据读取**：采用 **单字节分时读取策略**（而非连续突发读），提高软件 I2C 稳定性。
- 错误处理**：读取失败时返回 HAL_ERROR，防止脏数据进入算法。

3. 计步算法实现 计步算法部署于 Task_MPU6050 任务中（周期 20ms）：

- 数据预处理**：计算三轴加速度平方和 $Mag^2 = Ax^2 + Ay^2 + Az^2$ 。
- 静止基准校准**：上电后采集 20 组数据取平均，作为静止基准 $Base_{sq}$ 。
- 双阈值滞回比较**：

$$Threshold_{high} = Base_{sq} \times 1.0625$$

$$Threshold_{low} = Base_{sq} \times 1.0156$$

- 状态机逻辑**：引入 step_state (0: 等待上升沿, 1: 等待下降沿)，确保单次步伐仅计数一次。
- 时间窗防抖**：设置最小计步间隔 250ms，过滤高频噪声。

4 功能展示

4.1 功能描述

本模块实现了从硬件采集到算法处理再到数据输出的完整链路：

- 实时采集 MPU6050 加速度数据。
- 自动校准静止基准，自适应不同佩戴姿态。
- 动态识别步伐，更新全局步数变量。
- 支持通过蓝牙向上位机发送步数与时间信息。

4.2 演示结果与测试数据

表 1 计步算法测试数据（室内平地行走）

实际步数	系统计数	误差率
50	49	-2.0%
100	99	-1.0%
150	150	0.0%
200	198	-1.0%

5 物料与成本

5.1 物料清单

名称	数量	单价 (元)	用途
MPU6050 模块	1	7	六轴传感器
OLED 显示屏	1	12	数据显示
稳压模块	1	3	供电稳压
蓝牙模块	1	9	数据同步
EC11 编码器	1	19	菜单切换
充电模块	1	4	集成充电保护

5.2 项目总成本

$$7 + 12 + 3 + 9 + 19 + 4 = 54 \text{ 元}$$

5.3 损耗与损坏记录

- **MPU6050 硬件损坏与故障定位**: 项目初期采用 STM32 硬件 I2C 外设驱动 MPU6050, 调试过程中出现设备无响应、无法读取 WHO_AM_I 寄存器的现象。经逐一排查 (更换 I2C 引脚、降低时钟频率、检查上拉电阻), 确认原因为 MPU6050 模块内部失效 (推测为静电击穿或电源浪涌导致)。随后更换全新 MPU6050 模块, 并将驱动方案由硬件 I2C 重构为软件 I2C (GPIO 模拟时序), 彻底规避了硬件 I2C 的稳定性和兼容性风险。
- **硬件加固损耗**: 为提高系统抗震动能力, 对充电模块与接线的连接点进行了手工焊接加固, 过程中消耗少量焊锡与细导线, 无元器件损坏。

6 个人贡献

6.1 报告撰写分工

负责撰写课设报告中的成员二负责的相关部分内容。

6.2 个人编码工作内容

- 编写 `soft_i2c.c/h`: 软件 I2C 驱动 (开漏输出、动态切换、ACK 机制)。
- 编写 `mpu6050.c/h`: 传感器初始化、单字节读取、数据解析。
- 编写 `main.c` 中的 `Task_MPU6050`: 计步算法 (基准校准、双阈值、状态机)。
- 编写 FreeRTOS 任务创建代码 (`xTaskCreate`)。

6.3 硬件连接工作

- : 对硬件与接线交界处进行焊锡, 增加机械强度。

6.4 资本投入

无额外资金投入，使用团队公用物料。

6.5 开发中遇到的问题及解决方案

- **问题 1：硬件 I2C 频繁挂死。**解决：放弃硬件 I2C，改用软件 I2C，手动控制时序，彻底解决死锁。
- **问题 2：计步偶尔漏检或重复计数。**解决：引入双阈值滞回比较与时间窗防抖机制，优化状态机逻辑。
- **问题 3：硬件接触不良。**解决：手工焊接加固接口，消除松动隐患。

7 项目成果与反思

7.1 成果展示

成功实现了基于软件 I2C 的 MPU6050 数据采集，计步算法在常规场景下准确率高（误差 $< 2\%$ ），系统运行稳定，无死锁现象。完成了与 OLED、蓝牙模块的对接，为团队整体功能实现提供了数据支撑。

7.2 项目评估

- **成功之处：**
 - 软件 I2C 的成功移植，解决了 F103 硬件 I2C 的致命缺陷。
 - 计步算法兼顾了灵敏度和抗干扰能力。
 - 硬件焊接加固提升了系统可靠性。
- **不足之处：**
 - 计步算法对极端步态的情况（如手插兜、拖步）没有详细考虑。
 - 软件 I2C 速度相对硬件 I2C 较慢，虽不影响计步，但限制了高频数据传输。

7.3 个人反思与改进方向

- **反思：**在初期调试时过于依赖硬件外设，遇到 Bug 时应更早考虑软件替代方案。同时，硬件连接的牢固性应在项目初期就给予足够重视。
- **改进方向：**
 - 引入更复杂的滤波算法提升姿态估计精度。
 - 未来可考虑使用更高性能的 MCU 或专用计步芯片。

8 附录

8.1 设计图纸

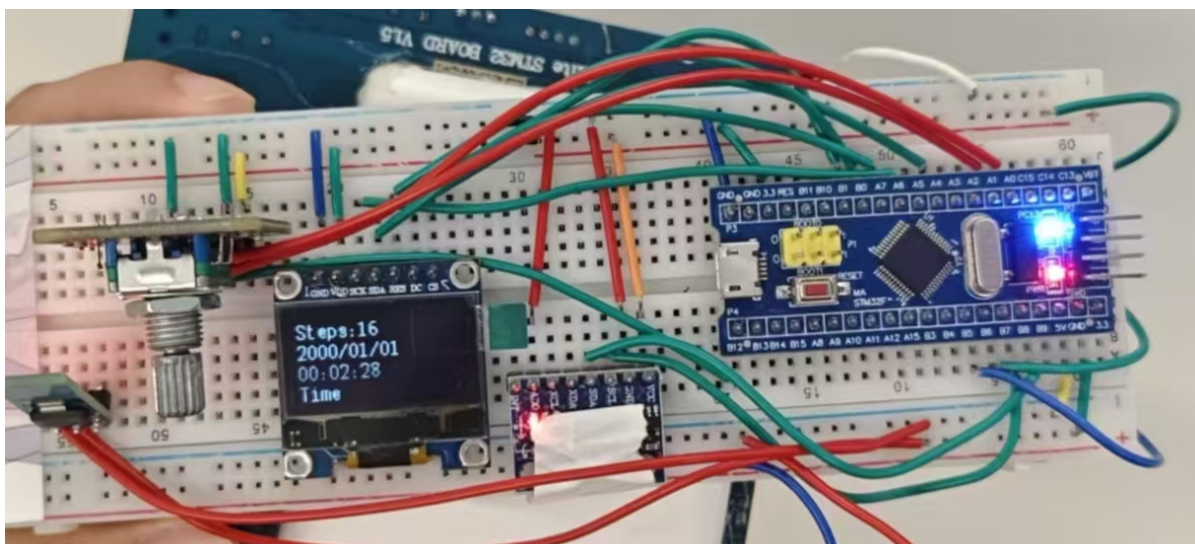


图 1 成品正面图

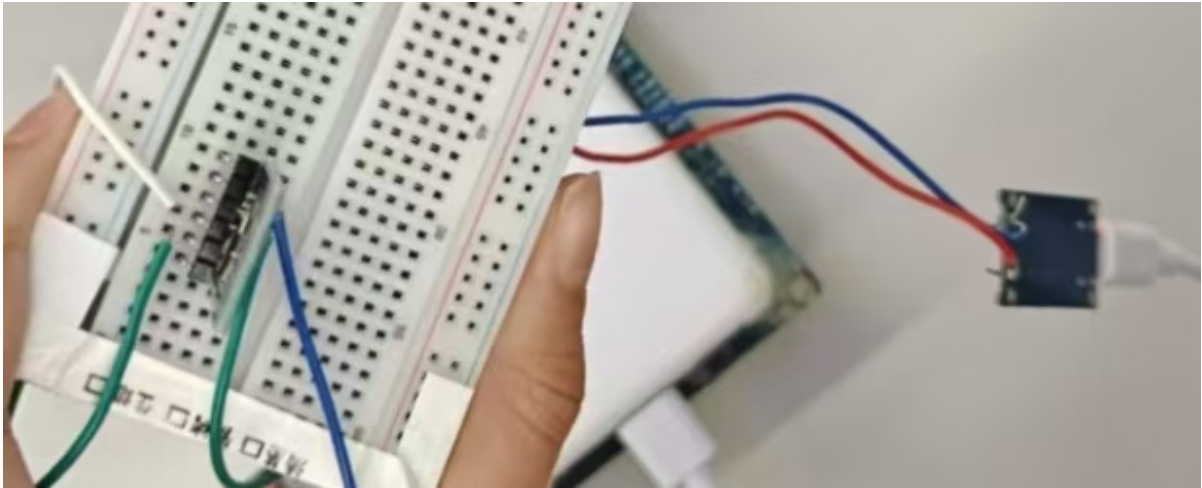


图 2 成品背面图

8.2 完整代码清单

- 软件 I2C 驱动：

- `soft_i2c.c` / `soft_i2c.h` ——实现 GPIO 模拟 I2C 时序，支持开漏输出、ACK/NACK 控制，用于驱动 MPU6050。

- MPU6050 传感器驱动：

- `mpu6050.c` / `mpu6050.h` ——包含初始化、寄存器读写、加速度数据解析，集成于 FreeRTOS 任务中。

- OLED 显示驱动：

- `oled.c` / `oled.h` ——基于 SPI 接口的 OLED 控制，支持字符、菜单显示。

- 编码器 (EC11) 驱动：

- `encoder.c` / `encoder.h` ——读取旋转差值与按键状态，供菜单任务使用。

- RTC 实时时钟驱动：

- `rtc.c` / `rtc.h` ——初始化并设置时间基准，用于蓝牙透传时间戳。

- UART 串口通信驱动：

- `usart.c` / `usart.h` ——配置 USART1 (波特率 115200)，用于蓝牙模块数据透传。

- **SPI 总线驱动:**

- `spi.c` / `spi.h` ——为 OLED 提供底层 SPI 通信支持。

- **GPIO 与中断配置:**

- `gpio.c` / `gpio.h` ——配置 EXTI 中断 (EC11 按键)、SPI/I2C 引脚复用。

- **TIM 定时器驱动:**

- `tim.c` / `tim.h` ——用于系统滴答、周期性触发 (如 HAL 时间基准)。

系统与应用层模块

- **主程序入口:**

- `main.c` ——系统初始化、外设配置、FreeRTOS 任务创建、全局变量定义。

- **菜单与交互逻辑:**

- `menu.c` / `menu.h` ——实现菜单状态机、显示刷新、编码器/按键响应。
- `font.h` ——内置字库, 供 OLED 显示使用。

- **FreeRTOS 内核配置:**

- `freertos.c` / `freertos.h` ——CubeMX 生成的 RTOS 初始化框架 (含默认任务)。
- `cmsis_os2.c` ——CMSIS-RTOS v2 封装层 (通常由 CubeMX 自动生成)。

- **STM32 HAL 库配置文件:**

- `stm32f1xx_hal_conf.h` ——HAL 库功能开关配置。
- `stm32f1xx_hal_msp.c` ——外设底层初始化 (如 GPIO、SPI、USART)。
- `stm32f1xx_it.c` / `stm32f1xx_it.h` ——中断服务函数 (如 EXTI、TIM)。
- `syscalls.c` / `sysmem.c` ——系统调用与内存管理 (RTOS 必需)。
- `system_stm32f1xx.c` ——系统时钟与启动代码。

头文件索引 (Inc/ 目录)

主要对外暴露的头文件包括:

- `main.h` —— 全局宏定义、外部变量声明。
- `soft_i2c.h` —— 软件 I2C 接口声明。
- `mpu6050.h` —— 传感器操作 API。
- `oled.h`, `encoder.h`, `menu.h`, `rtc.h`, `usart.h`, `spi.h`, `tim.h`, `gpio.h` —— 各外设驱动接口。
- `FreeRTOSConfig.h` —— FreeRTOS 配置参数 (如任务优先级、堆栈大小)。

8.3 参考文献

- STM32F10xxx Reference Manual
- MPU6050 Datasheet
- FreeRTOS Reference Manual